

Getting things done with Airflow

github.com/datamindedacademy/workflows_with_airflow

Orchestrating work with Apache Airflow · Data Minded Academy



Contents

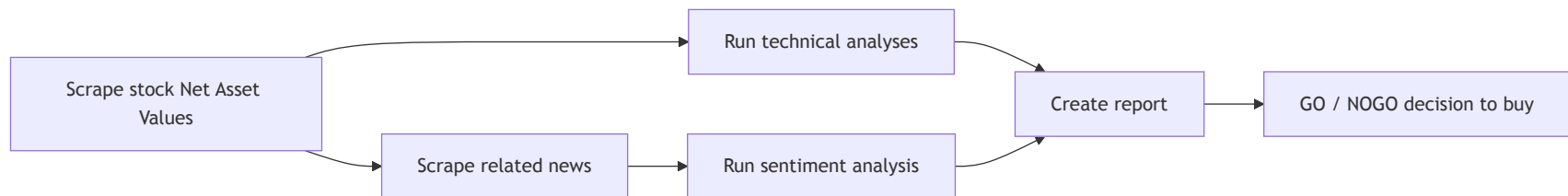
01	Orchestration
02	Architecture
03	The web UI
04	Building a DAG
05	Scheduling
06	Templating
07	DAG design patterns
08	Operators & trigger rules
09	Cross-DAG dependencies
10	Sharing data & configuration
11	Best practices
12	Wrap-up



Orchestration



Why a scheduler? A workflow triggered at a definite time



Run daily, at 08h, but not in the weekends.



What is Airflow, and why?

A workflow scheduler for batch jobs, originally built at Airbnb, now mostly maintained by Astronomer. **Airflow 3** is the current major version — this course runs 3.1.

- Open-source automation of batch workflows
- Workflows are **Python code**, with its whole ecosystem available
- Kept in version control, deployed through CI/CD
- Extend it with your own operators, hooks and plugins
- Ready-made operators for Spark, SQL, Kubernetes, the big clouds, ...
- A large community, so most problems are already answered somewhere
- A UI that tells you what ran, what failed, and how long it took



The Airflow landscape: modern competitors

Airflow 3 closed much of this gap — **Assets** answer software-defined assets, and the **Task SDK** answers the developer-experience critique. Also worth knowing: Kestra, Temporal, Flyte.

Prefect

Developer-centric

Dynamic, modern data stacks with native Python integration and less boilerplate.

- "Code as workflows"
- Dynamic task mapping
- Scalable hybrid-cloud agent model

Dagster

Data-asset centric

Focuses on data assets rather than just tasks; strong local dev & testing.

- Built-in data lineage
- Rich UI for asset monitoring
- Software-defined assets

Argo Workflows

Cloud-native (K8s)

Open-source, container-native workflow engine as a Kubernetes CRD.

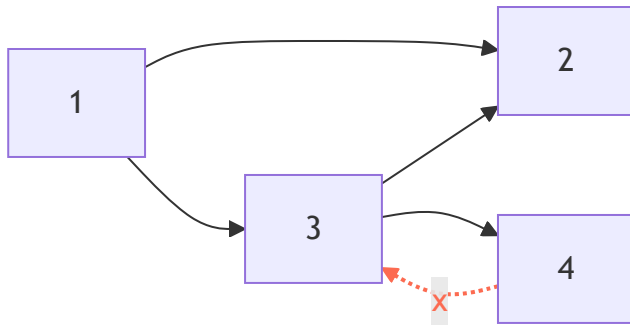
- Native Kubernetes execution
- YAML configuration
- Great for heavy ML/container jobs



How would you automate a
sequence of tasks?



Directed Acyclic Graphs allow **ordering**



Valid execution orders for this DAG, with edges meaning "before":

1 → 3 → 2 → 4

1 → 3 → 4 → 2



Not acyclic? Not allowed.

Cycles have no valid execution order.



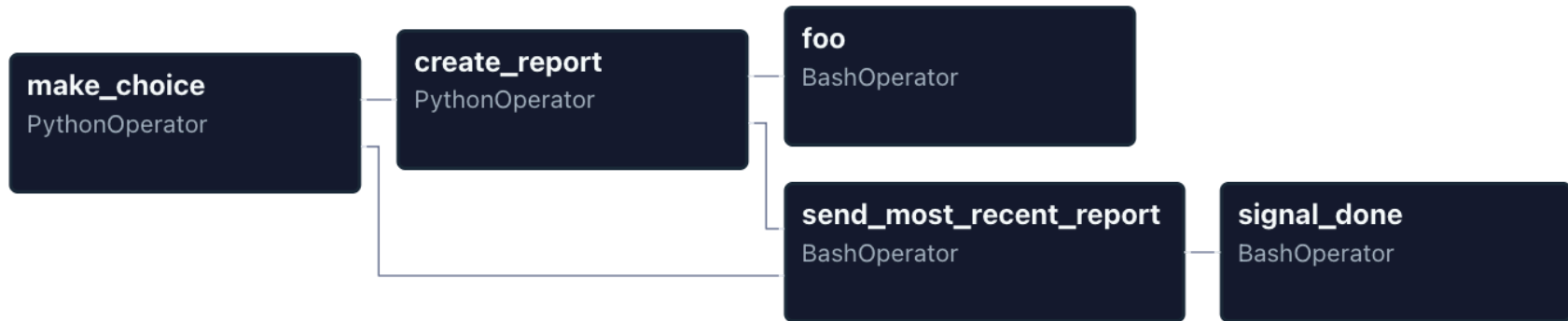
A workflow describes the **what**, not the **how**



- **Steps**, in a defined **order**
 - A **graph**: entities with relationships — nodes & edges
 - **Direction** between the nodes
- **Acyclic**: no revisiting nodes
 - Stays **high-level**: it describes the *what*, not the *how*



Example DAG in Airflow with 5 tasks



Tasks come in all shapes and sizes

- Train a model on a GPU
 - Query an API
 - Execute a Spark job
 - Execute ML inference
 - Load, clean and store a dataframe
 - Read from an API and store in a bucket
- Read from a bucket and store in Snowflake
 - Create a PDF
 - Execute SQL on Snowflake
 - Send an email
 - Call GPT-4
 - Run a Docker container
 - Join 2 parquet tables and aggregate



Exercise 0

0 · hello airflow

Intro to the Airflow UI and running your first DAG

`0_hello_airflow`



Architecture



Why Airflow 3?

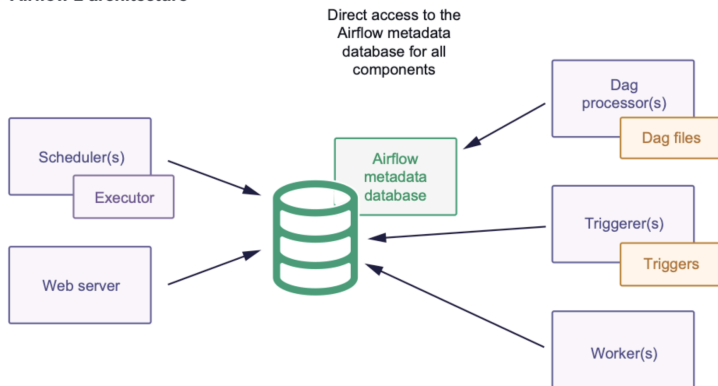
- **Decoupled architecture** — the Task SDK means workers no longer need full DAG file access
 - **Performance boost** — removes "parsing loop" bottlenecks for near-instant task startup
 - **Massive scalability** — designed for millions of daily tasks with a lighter footprint
- **Data-first design** — first-class support for Data Assets and event-driven scheduling
 - **Enhanced DevEx** — a modernized UI and improved local development experience



Airflow 2 vs. Airflow 3

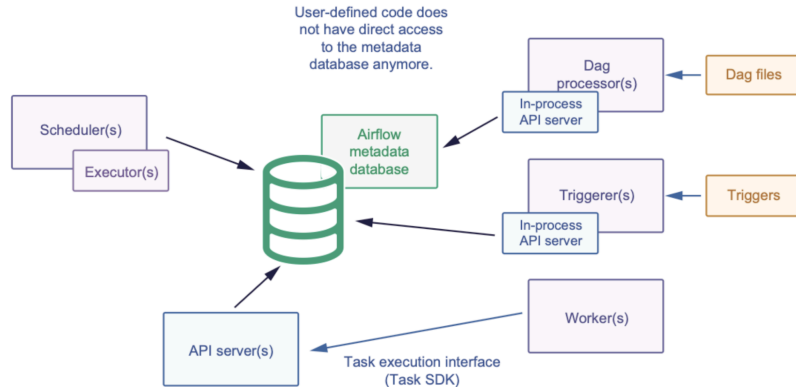
Airflow 2

Airflow 2 architecture



Airflow 3

Airflow 3 architecture



In Airflow 3, user-defined code no longer has direct access to the metadata database — everything goes through the API server / Task SDK.



The components you actually **deploy**

- **API server** — serves the REST API *and* the UI. Called the *webserver* in Airflow 2
- **DAG processor** — parses your DAG files. A **required, standalone** process in Airflow 3, so the scheduler never touches your code
- **Scheduler** — decides which task instances may run, and hands them to the executor
- **Executor** — starts workers: Local, Celery, Kubernetes, Edge
- **Workers** — run the tasks, possibly on other machines
- **Triggerer** — *optional*; runs deferred tasks in an asyncio loop (see section 9)
- **Metadata database** — task status, runtime, configuration, ...



Keep the DAG processor cheap. It re-parses every DAG file on a loop, so anything expensive at the top level of your file is paid over and over — see exercise 1.



Which of those you actually touch

Authoring

Write Python files into the **DAG bundle** — for us, a folder; in production usually a git repository the DAG processor syncs.

Operating

The **UI**, served by the API server and backed by the metadata database. Trigger, inspect, clear, backfill.

Rarely

Scheduler, executor and workers are infrastructure. The CLI and scheduler logs are for when something is genuinely wrong.



Airflow 3 tracks the **version** of the DAG each run used. A run finishes against the code it started with, even if you deploy mid-run — and the UI shows you which version that was.



The web UI



Web UI: DAGs view

The screenshot displays the Apache Airflow Web UI DAGs view. The interface includes a sidebar with navigation links (Home, DAGs, Assets, Browse, Admin, Security) and a main content area. The main content area has tabs for 'Dags', 'Dag Runs', and 'Task Instances'. Below the tabs is a search bar and a filter section with buttons for 'All', 'Failed', 'Queued', 'Running', 'Success', and 'Required Actions'. The filter section also includes a 'Filter by tag' dropdown and an 'All' dropdown. The main content area shows a list of 14 DAGs. The table has columns for 'Dag ID', 'Schedule', 'Next Run', 'Latest Run', and 'Tags'. The DAGs listed are:

Dag ID	Schedule	Next Run	Latest Run	Tags
Hello Airflow	1 day, 0:00:00	2026-02-03 09:37:13	2026-02-02 09:37:13	
solution_1_investment_analysis	@once		2025-01-01 01:00:00	
solution_2_happy_birthday_python_v3	0 13 8 8 *	2026-05-09 01:00:00	2025-05-09 01:00:00	
solution_2_happy_birthday_v1	0 0 4 8 *	2026-08-04 02:00:00	2025-08-04 02:00:00	
solution_2_happy_birthday_v2	0 0 4 8 *	2026-08-04 12:00:00	2025-08-04 12:00:00	



Web UI: grid view

The screenshot displays the Apache Airflow web interface in grid view. The top navigation bar includes links for Home, DAGs, Assets, Browse, Admin, and Security. The main header shows the DAG name 'solution_8_ignoring_failure' and the specific run 'backfill_2026-01-31T00:00:00+00:00'. A blue banner indicates 'Backfill in progress: 2026-01-26 11:11:00 - 2026-02-02 11:11:00'. On the left, a task list shows 'task0' through 'task6', with 'task4' selected. A central grid visualization shows the execution timeline of tasks, with a tooltip for 'Task ID: task4' showing 'State: Success'. The right panel provides details for the selected DAG run, including a 'Task Instances' table.

DAG Run Details:

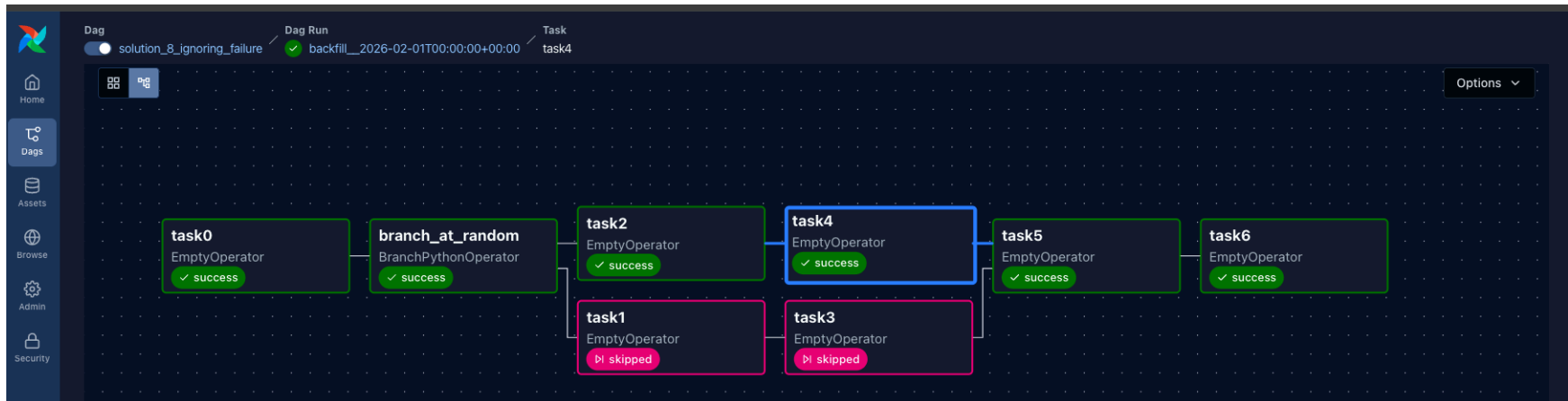
- DAG: `backfill_2026-01-31T00:00:00+00:00` (Success)
- Logical Date: 2026-01-31 01:00:00
- Run Type: backfill
- Start Date: 2026-02-02 11:11:40
- End Date: 2026-02-02 11:11:46
- Duration: 00:00:05.838
- Triggering User Name: airflow
- Dag Version(s): v2

Task Instances Table:

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
task6		Success	2026-02-02 11:11:45	1	EmptyOperator	
task5		Success	2026-02-02 11:11:44	1	EmptyOperator	
task3		Success	2026-02-02 11:11:43	1	EmptyOperator	
task1		Success	2026-02-02 11:11:42	1	EmptyOperator	
task4		Skipped	2026-02-02 11:11:42	0	EmptyOperator	
task2		Skipped	2026-02-02 11:11:41	0	EmptyOperator	
branch_at_random		Success	2026-02-02 11:11:41	1	BranchPythonOperator	00:00:00.073
task0		Success	2026-02-02 11:11:40	1	EmptyOperator	



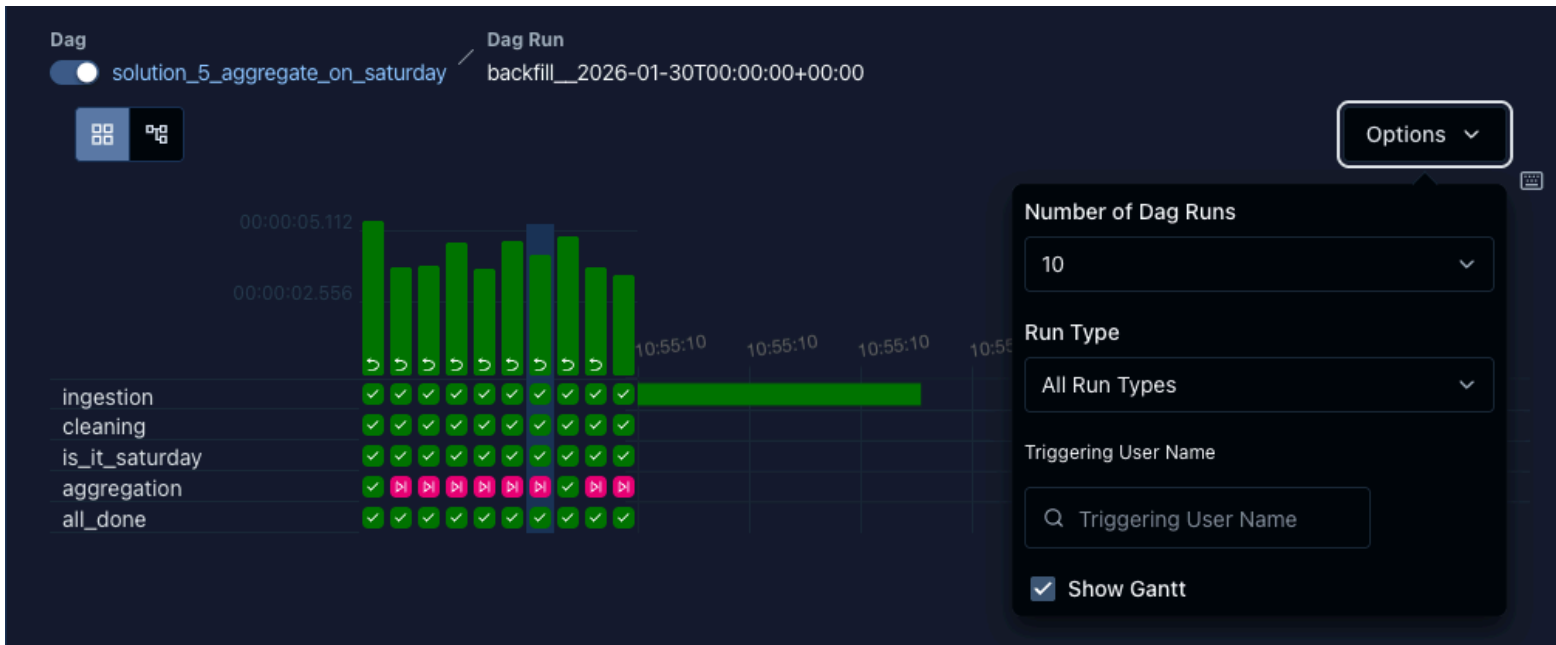
Web UI: graph view



Great in development, because a picture says more than a thousand words (of Python 🐍).



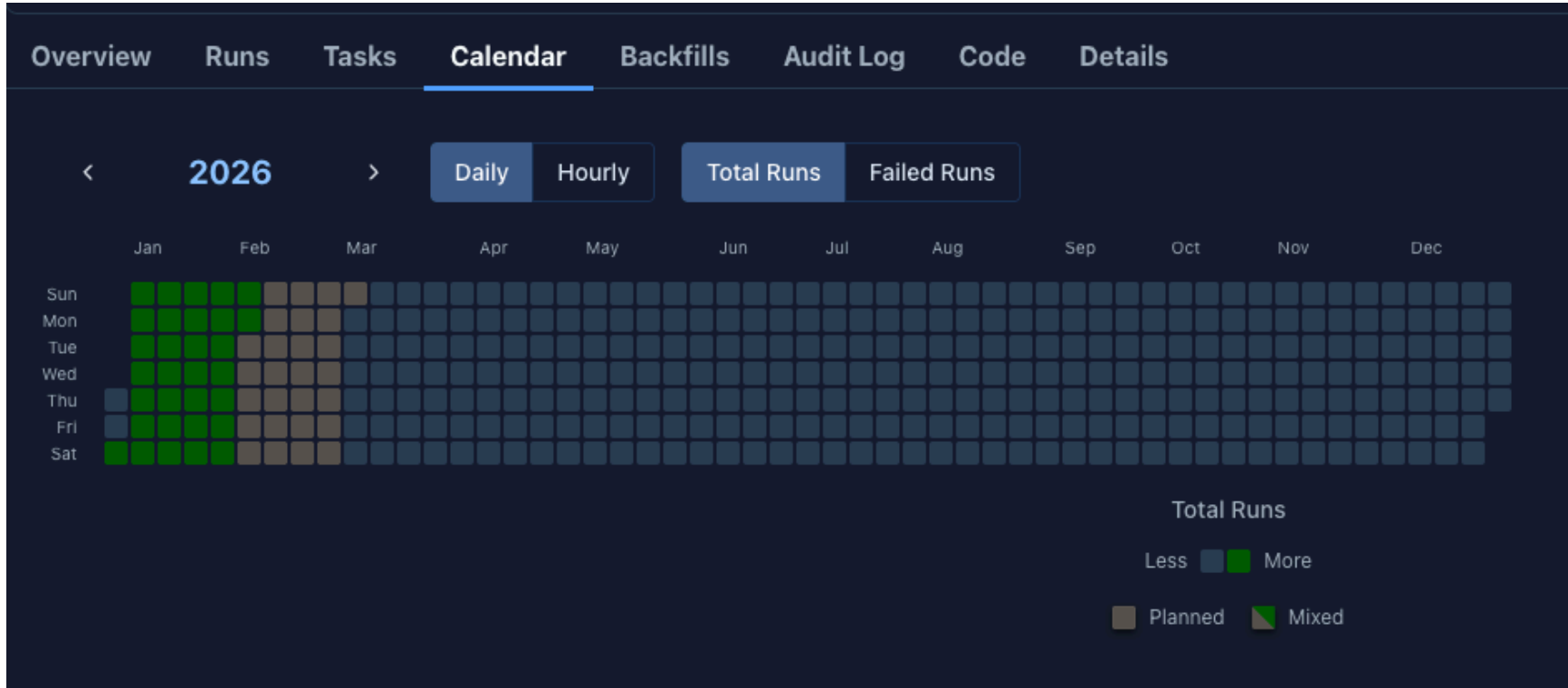
Web UI: Gantt chart



Good for observing the duration of a specific task run, and parallelism within a DAG (⚠️ parallelism is configurable).
Less ideal for task dependencies — use the Graph tab for that.



Web UI: calendar view



Web UI: code view

A read-only view on the code behind the workflow — and in Airflow 3, on the **exact version each run used**, since a run completes against the code it started with.

The screenshot shows the 'Code' tab in the Airflow web interface. At the top, there's a header for the DAG 'solution_5_aggregate_on_saturday'. Below this, there are tabs for Overview, Runs, Tasks, Calendar, Backfills, Audit Log, and Code (which is selected). The code is displayed in a dark-themed editor. It starts with imports for datetime, DAG, BashOperator, and BranchDayOfWeekOperator. A comment indicates a bug in the DAG's behavior on Saturdays. The code defines a DAG with a daily schedule and a task named 'ingestion'. It also includes logic for a 'cleaning' task and a branch operator to handle different days of the week.

```

1  import datetime as dt
2
3  from airflow import DAG
4  from airflow.providers.standard.operators.bash import BashOperator
5  from airflow.providers.standard.operators.empty import EmptyOperator
6  from airflow.operators.weekday import BranchDayOfWeekOperator
7
8  """
9  Exercise 5
10
11  This DAG needs to do an extra step on Saturday,
12  but the current implementation has some downsides.
13
14  What's wrong? And how can you fix this?
15  """
16
17  dag = DAG(
18      dag_id="solution_5_aggregate_on_saturday",
19      description="On Saturdays we run aggregations",
20      default_args={"owner": "Airflow"},
21      schedule="daily",
22      start_date=dt.datetime(2025, 1, 1),
23      end_date=dt.datetime(2026, 1, 1),
24  )
25
26
27  def create_task(name):
28      return BashOperator(
29          task_id=name,
30          dag=dag,
31          bash_command=f"echo '{name} done'",
32      )
33
34
35  ingestion_task = create_task("ingestion")
36  cleaning_task = create_task("cleaning")
37  aggregation_task = create_task("aggregation")
38  all_done = EmptyOperator(task_id="all_done", dag=dag, trigger_rule="all_done")
39
40  branch_for_saturday = BranchDayOfWeekOperator(
41      task_id="is_it_saturday",
42      dag=dag,
43      use_task_logical_date=True,
44      follow_task_id_if_true=aggregation_task.task_id,
45      follow_task_id_if_false=all_done.task_id,
46      week_days="saturday",
47  )

```



Building a DAG



The anatomy of a DAG: schedule, tasks, operators

- Each DAG has a schedule and a unique `dag_id`
- Each DAG has at least one task
- Each task belongs to a DAG and has a unique `task_id`
- Each task is an instance of an operator
- Tasks run only when all upstream tasks succeeded by default, but this is configurable via `trigger_rule` (e.g. `trigger_rule="one_failed"`)
- Many operator types exist: `BashOperator` , `PythonOperator` , `KubernetesPodOperator` , `SSHOperator` , ... and you can make your own

```
with DAG(dag_id="example", ...):  
    BashOperator(  
        task_id="example",  
        bash_command="date",  
        trigger_rule="all_success",  
    )
```



A workflow is defined by the DAG class and its operators

```
with DAG(
    dag_id="reporting",
    schedule="@daily",
    start_date=pendulum.datetime(2026, 1, 1, tz="Europe/Brussels"),
    default_args={"retries": 1}, # passed to every operator, overridable per task
) as dag:
    retrieve = PythonOperator(task_id="retrieve", python_callable=retrieve_file)
    process = PythonOperator(task_id="process", python_callable=process_data)
    store = PythonOperator(task_id="store", python_callable=store_data)

    retrieve >> process >> store
```

`dag_id` s and `task_id` s must be unique so you can reference them (e.g. from an `ExternalTaskSensor`). 💡 Don't repeat strings all over the place — imagine fixing a typo. IDEs can autorename objects, but not standalone strings.



The Python Operator executes a Python callable on a worker

```
import pendulum
from airflow.sdk import DAG
from airflow.providers.standard.operators.python import PythonOperator

def say_hello():
    print("Hello Airflow")
    return "this goes to xcom"

with DAG(
    dag_id="hello_airflow",
    schedule="@daily",
    start_date=pendulum.datetime(2026, 1, 1, tz="Europe/Brussels"),
) as dag:
    task = PythonOperator(
        task_id="hello_world",
        python_callable=say_hello,
    )
```

`airflow.sdk` is the Airflow 3 public interface — the Task SDK. It is what workers get, and it is where `DAG`, `task`, `chain`, `Asset` and friends now live.



Two ways to write the same task: pick one, stay consistent

Classic: PythonOperator

```
def extract():
    return {"value": 42}

with DAG(dag_id="example") as dag:
    task = PythonOperator(
        task_id="extract",
        python_callable=extract,
    )
```

TaskFlow API

```
from airflow.sdk import dag, task

@dag(dag_id="example")
def example():
    @task
    def extract():
        return {"value": 42}

    extract()

example()
```



It is a trade-off, not a right answer

Mixing both in one repository is the only choice that is clearly wrong.

- **For TaskFlow:** far less boilerplate; XComs are just return values and arguments; `.expand()` for dynamic mapping reads naturally; it is where Airflow 3's Task SDK is heading, and what the docs recommend
- **Against:** decorators are not beginner friendly; it looks unlike your other operators, so a mixed DAG reads inconsistently; the bare `extract()` on the last line suggests the scheduler runs your function, which is not what happens
- **Either way:** non-trivial Python belongs in a package you import, not in the DAG file



For the record, the docs have a favourite

PythonOperator

Use the `@task` decorator to execute Python callables.

Warning

The `@task` decorator is recommended over the classic `PythonOperator` to execute Python callables.

airflow/example_dags/example_python_operator.py

[\[source\]](#)

```
@task(task_id="print_the_context")
def print_context(ds=None, **kwargs):
    """Print the Airflow context and ds variable from the context."""
    pprint(kwargs)
    print(ds)
    return "Whatever you return gets printed in the logs"

run_this = print_context()
```

This course uses the classic operator in most exercises because it makes the machinery visible — exercise 10 gives you both. Once you know what a decorator hides, choose deliberately for your own team.



Exercise 1

1 · investing

Top-level code cost — why heavy work at import time slows the scheduler

```
1_investing
```



Scheduling



Five ways to tell Airflow **when to run**

Method	Example
Presets	<code>None</code> , <code>@once</code> , <code>@hourly</code> , <code>@daily</code> , <code>@weekly</code> , <code>@monthly</code> , <code>@yearly</code>
Cron syntax	<code>* / 5 1,2 * * *</code>
<code>datetime.timedelta</code>	<code>datetime.timedelta(days=4)</code>
Timetable	a schedule object — the only way to get a data interval , or a fully custom calendar
Asset	each time another process updates an Asset

Every preset is just cron in disguise — which is why they align to the start of a calendar unit:

- `@hourly` $\equiv$ `0 * * * *`
- `@daily` $\equiv$ `0 0 * * *`
- `@weekly` $\equiv$ `0 0 * * 0` (Sunday 00:00)
- `@monthly` $\equiv$ `0 0 1 * *`
- `@yearly` $\equiv$ `0 0 1 1 *`
- `None` / `@once` have no cron equivalent



Airflow scheduling: **cron** syntax

```

minute (0 - 59)
hour (0 - 23)
day of the month (1 - 31)
month (1 - 12)
day of the week (0 - 6) (0: Su, 6: Sa)
* * * * *
```

Mnemonic: **Mi Ho Da Mo We**.

- `*/5 1,2,3 * * *` — every 5th minute of the 1st, 2nd and 3rd hour
- `*/5 1-3 * * *` — same as above
- `*/30 * * * 0` — every half hour on Sunday
- `0 8 * * 1-5` — 08h00, Monday through Friday

Use tools like crontab.guru to verify the expression matches your intent.



In Airflow 3, a cron schedule fires **at** the cron time

Give `schedule` a cron string or a preset and you get a `CronTriggerTimetable`: the run fires the moment the cron expression matches. There is no window — `data_interval_start`, `data_interval_end` and `logical_date` are all the same instant.

```
with DAG(dag_id="birthday", schedule="0 0 3 8 *",      # Mi Ho Da Mo We → 00:00 on 3 August
        start_date=pendulum.datetime(1987, 8, 3, tz="Europe/Brussels")):
    ...
```

Run fires at	logical_date	{{ ds }}
1987-08-03 00:00	1987-08-03 00:00	1987-08-03
1988-08-03 00:00	1988-08-03 00:00	1988-08-03



This changed in Airflow 3: `create_cron_data_intervals` now defaults to `False`. In Airflow 2 the same DAG fired a year **later**, at the end of an interval.



Presets align to the calendar unit, not to your start_date

Say you want a birthday DAG for someone born on **1987-08-03**, and you reach for `@yearly` :

```
with DAG(dag_id="birthday", schedule="@yearly",
         start_date=pendulum.datetime(1987, 8, 3, tz="Europe/Brussels")):
    ...
```

	@yearly	0 0 3 8 *
Cron it resolves to	0 0 1 1 *	0 0 3 8 *
Fires on	1 January ❌	3 August ✅



A preset never looks at your `start_date` — it only decides when the DAG may **start** firing. Write the cron out when you need a specific day.

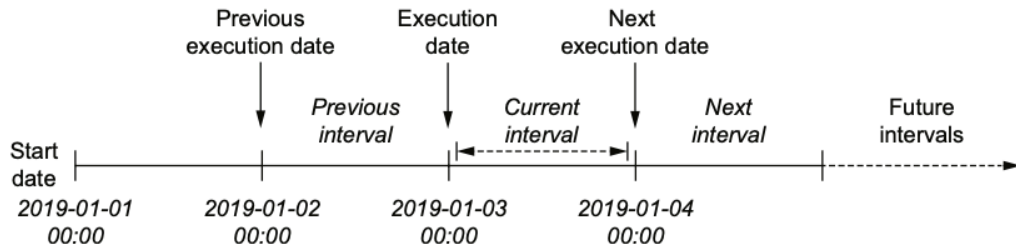


Summarising a period? Opt in to a data **interval**

Fine for "send the birthday mail". Not enough for "sum yesterday's revenue" — that needs a **window**. Pass a timetable explicitly: the run then fires at the **end** of its interval, with both edges templated.

```
from airflow.timetables.interval import CronDataIntervalTimetable

with DAG(dag_id="daily_revenue",
        schedule=CronDataIntervalTimetable("0 0 * * *", "Europe/Brussels"),
        start_date=pendulum.datetime(2026, 1, 1, tz="Europe/Brussels")):
    ...
```



The diagram's *execution date* is what Airflow 3 calls `logical_date` — with this timetable it is the interval **start**, so `{{ ds }}` is the day being summarised, not the day the DAG runs.



Careful: intervals are not always the same length

January 2020						
Mo	Tu	We	Th	Fr	Sa	Su
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Give that timetable a business-day cron — `0 0 * * 1-5` — and the weekend disappears, so Friday's interval is **three days long**:

data_interval_start	data_interval_end	Span
2020-01-02	2020-01-03	1 day
2020-01-03	2020-01-06	3 days

A query that assumes "one interval = one day" quietly triple-counts every Friday. Always bracket on `data_interval_start` / `data_interval_end` rather than on `{{ ds }}` plus one day.



Reading Airflow 2 DAGs: what changed

Airflow 2	Airflow 3	
<code>schedule_interval=</code>	<code>schedule=</code>	renamed
<code>execution_date</code>	<code>logical_date</code>	removed from the context
<code>{{ next_ds }}</code> , <code>{{ prev_ds }}</code> , <code>{{ tomorrow_ds }}</code> , ...	<code>{{ data_interval_end ds }}</code>	removed
<code>logical_date</code> $\equiv$ <code>data_interval_start</code>	<code>logical_date</code> $\equiv$ <code>run_after</code>	same name, new meaning
cron $\Rightarrow$ data interval	cron $\Rightarrow$ single point in time	<code>create_cron_data_intervals=False</code>
<code>catchup=True</code> by default	<code>catchup=False</code> by default	see next section



The fourth row is the dangerous one: old DAGs still import and run, but `logical_date` now means something else. Re-read every date-sensitive task when you migrate.



Exercise 2

2 · birthday

Scheduling basics: presets vs. cron, aligning `start_date`, `catchup=False`

`2_birthday`



Templating



Python interpolates at **import** time — that's the problem

Plain Python f-strings

```
weight = 7000
print(f"Today's weight is {weight} g")

output = f"Today's weight is {weight / 1000} kg"
print(output)
```

The value is baked in the moment the line is evaluated.

A DAG file is evaluated constantly

```
sql = f"""
SELECT SUM(amount) FROM {table}
WHERE dt BETWEEN
    '{dt.date.today() - dt.timedelta(days=1)}'
    AND '{dt.date.today()}'
"""
```

`dt.date.today()` is the **parse** date, not the run date.
Reruns silently query the wrong days.



Mixing languages in one file also costs you syntax highlighting, autocompletion, and scheduler CPU on meaningless compute.



Jinja templates defer the value until **execution** time

dags/sql/daily_revenue.sql

```
SELECT SUM(amount)
FROM {{ params.table }}
WHERE transaction_timestamp
    BETWEEN '{{ data_interval_start | ds }}'
    AND '{{ data_interval_end | ds }}'
```

dags/daily_revenue.py

```
from airflow.providers.common.sql.operators.sql \
    import SQLExecuteQueryOperator

with DAG(
    dag_id="daily-revenue",
    schedule=CronDataIntervalTimetable(
        "0 0 * * *", "Europe/Brussels"),
    start_date=pendulum.datetime(2026, 1, 1,
        tz="Europe/Brussels"),
) as dag:
    revenue = SQLExecuteQueryOperator(
        task_id="query_revenue",
        conn_id="postgres_default",
        sql="sql/daily_revenue.sql",
        params={"table": "SALES"},
    )
```

This is what makes a DAG **idempotent**: rerun Saturday's run on Monday and it still reports on Saturday.



Macros let you call Python functions inside a template

```
PythonOperator(
    task_id="example_python",
    python_callable=myprint,
    op_args=[
        "Day of week: "
        "{{ logical_date.format('dddd') }}",
        "Task id: {{ task_instance.task_id }}",
        "Days since start: {{ macros.dateutil"
        "'.relativedelta.relativedelta("
        "data_interval_start, dag.start_date).days }}",
        "Environment: "
        "{{ var.value.get('environment', 'test') }}",
    ],
)
```

`macros.` exposes Python inside Jinja; `var.value.` reads the Variables you defined in the UI. You can register your own macros on the DAG.

Task Instance: example_python at 2000-01-06, 00:00:00



Task Instance Details

< > Rendered Template

Log

XCom

Rendered Template

templates_dict

op_args

```
1 [
2     "Execution day of week: Thursday",
3     "Schedule interval: @daily",
4     "Current task id: example_python",
5     "Days between logical date and start of workflow: 5",
6     "Current environment: test"
7 ]
```

op_kwargs

```
1 {}
```

The **Rendered Template** tab on a task instance shows exactly what the worker received.



Exercise 3

3 · birthday_full

Templating with Jinja: `data_interval_end` (logical date) vs. wall-clock time

`3_birthday_full`



DAG design patterns



Catchup and backfills

catchup=False — the Airflow 3 default

Nothing older than now is scheduled. Deploy a DAG with a `start_date` two years back and you get **one** run, not seven hundred.

- What you want while developing
- Backfill deliberately, when you mean to

catchup=True — opt in

On deploy, Airflow schedules **every** missed period between `start_date` and now.

- Only safe if your DAG is genuinely idempotent
- Watch `max_active_runs`, or you flood your warehouse



Reversed in Airflow 3: `catchup_by_default` is now `False`. Airflow 2 caught up unless you told it not to — a classic first-deploy surprise.



A backfilled run carries the **logical date of the period it stands for**, not today. Code that calls `datetime.now()` quietly produces today's answer for last year's data.



Task chaining **shorthands**

```
# Simple chain
a >> b >> c

# Fan-out / fan-in
a >> [b, c] >> d

# Cross-downstream: every task in the first list feeds every task in the second
from airflow.sdk import cross_downstream
cross_downstream([a, b], [c, d, e])

# chain() strings lists together – the one thing >> cannot do
from airflow.sdk import chain
chain(a, [b, c], [d, e], f)
```

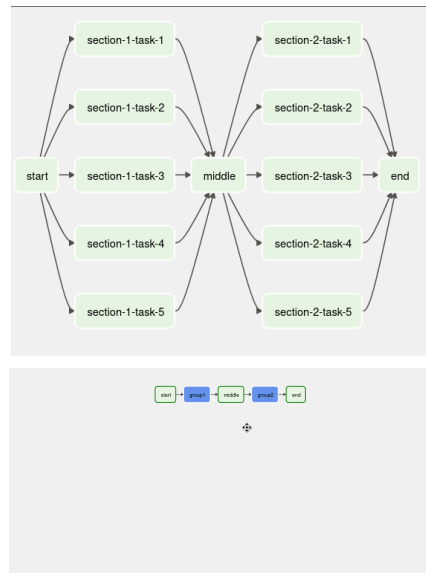


Task groups hide complexity

```
from airflow.sdk import TaskGroup

indices = range(1, 6)
with dag:
    start, middle, end = (
        EmptyOperator(task_id=s)
        for s in ("start", "middle", "end")
    )
    with TaskGroup(group_id="group1") as group1:
        [EmptyOperator(task_id=f"section-1-task-{n}")
         for n in indices]
    with TaskGroup(group_id="group2") as group2:
        [EmptyOperator(task_id=f"section-2-task-{n}")
         for n in indices]

    start >> group1 >> middle >> group2 >> end
```



Collapsed in the graph view, expandable on click — the DAG behaves identically.



When the list is only known at **run time**

A for-loop is resolved at parse time

```
for table in ["sales", "users"]:
    PythonOperator(
        task_id=f"ingest_{table}",
        python_callable=ingest,
        op_args=[table],
    )
```

The DAG processor must know the list. It cannot depend on anything a task produces.

Dynamic task mapping is resolved at run time

```
@task
def list_tables() -> list[str]: ...

@task
def ingest(table: str): ...

ingest.expand(table=list_tables())
```

Airflow creates one mapped task instance per element **after** `list_tables` runs.



Rule of thumb: the list comes from config or code → for-loop. The list comes from a database, an API or a bucket listing → `.expand()`. Exercise 10 does both.



Exercises 4 & 5

4 · saturday

Catchup & backfills: `catchup=True`, logical date
vs. `datetime.now()`, skipped vs. failed

4_saturday

5 · repetition

DRY DAGs — generating tasks and dependencies
programmatically instead of copy-pasting

5_repetition



Operators & trigger rules



Some operators come with Airflow. Others are optional — or **custom**

Standard provider — always there

`airflow.providers.standard.operators`

- `BashOperator` — executes a bash command
- `PythonOperator` — calls a Python function
- `BranchPythonOperator` — picks the next `task_id`
- `ShortCircuitOperator` — stops a branch on a condition
- `EmptyOperator` — a no-op join / marker

Other provider packages

- `SQLExecuteQueryOperator` — one operator for every SQL backend
- `HttpOperator`
- `SSHOperator` — run commands on another server
- `DockerOperator` , `KubernetesPodOperator`
- Many more in the [providers packages listing](#)



Airflow 3 moved the built-ins into the **standard provider** and retired the per-database operators (`PostgresOperator` → `SQLExecuteQueryOperator`). Old imports simply fail.



An operator is a class — and you can read the source

```
class BaseOperator(Operator, LoggingMixin, ...):  
    """Abstract base class for all operators.""" # has an execute() method  
  
class BashOperator(BaseOperator): ... # a plain leaf operator  
  
class BaseBranchOperator(BaseOperator, SkipMixin): ...  
class BranchDayOfWeekOperator(BaseBranchOperator): ... # picks a downstream task_id
```

A task is an **instance** of an operator; the operator is a template — missing details, but fit for a purpose. Everything inherits from `BaseOperator` and implements `execute()`.



Deferred execution → distributed computing. When an operator surprises you, open its class: it is ordinary Python.



Should you use the most specific operator?

Provider packages?

Depending on externally defined operators to execute logic creates tight coupling between your orchestration tool and the tasks you want to run: an operator inherently combines orchestration bugs with execution bugs, requires all used components to be compatible, and promotes system lock-in.



Airflow environments should be lean. Standardize on a limited set of operators suited to generic tasks — containers are a good fit. A task created with `HttpOperator` could equally run via `BashOperator` or `PythonOperator`. Trade-off: atomicity vs. level of abstraction.



Trigger rules: customize when to trigger a task

Optional argument on every operator. By default a task starts once all of its upstream tasks completed successfully — but some scenarios need a different rule.

<code>all_success</code>	(default) all parents have succeeded
<code>all_failed</code>	all parents are in a <code>failed</code> or <code>upstream_failed</code> state
<code>all_done</code>	all parents are done with their execution, whatever the outcome
<code>one_failed</code>	fires as soon as at least one parent has failed; does not wait for the rest
<code>one_success</code>	fires as soon as at least one parent succeeds; does not wait for the rest
<code>none_failed</code>	no parent failed, i.e. all parents succeeded or were skipped
<code>none_skipped</code>	no parent is in a <code>skipped</code> state
<code>always</code>	dependencies are just for show — trigger at will

♥ Type safety? → `from airflow.utils.trigger_rule import TriggerRule`, then `TriggerRule.ALL_DONE`.



Exercises 6 & 7

6 · failing_tasks

Trigger rules — `ALL_SUCCESS` vs. `ALL_DONE` / `ONE_SUCCESS`, letting downstream work survive an upstream failure

`6_failing_tasks`

7 · ignoring_failure

Trigger rules combined with retries & branching

`7_ignoring_failure`



Cross-DAG dependencies



Cross-DAG dependencies: use Trigger/Sensor

You cannot create a direct dependency between tasks in two different DAGs. Mimic the dependency, in either direction:

Pull: ExternalTaskSensor

```
from airflow.providers.standard.sensors \
    .external_task import ExternalTaskSensor

ExternalTaskSensor(
    task_id="wait_for_ingest",
    external_dag_id="ingest",
    external_task_id="store",
)
```

A sensor waits. Needs matching schedules on both DAGs.

Push: TriggerDagRunOperator

```
from airflow.providers.standard.operators \
    .trigger_dagrun import TriggerDagRunOperator

TriggerDagRunOperator(
    task_id="start_child",
    trigger_dag_id="child",
    logical_date="{{ ds }}",
    wait_for_completion=True,
)
```

With `wait_for_completion`, this task ends when the child does.

Both couple two DAGs by **schedule**. The third option — Assets — couples them by **data** instead, and is usually the better answer.



A waiting task should not hold a worker slot



Naive: poke in a loop

```
ExternalTaskSensor(
    task_id="wait",
    external_dag_id="ingest",
)
```

Occupies a **worker slot** for the whole wait. Sixteen sensors waiting six hours each will deadlock a small cluster.



Deferrable

```
ExternalTaskSensor(
    task_id="wait",
    external_dag_id="ingest",
    deferrable=True,
)
```

Releases the slot and hands the wait to the **triggerer**, which polls thousands of these in one asyncio loop.

zzz

`mode="reschedule"` is the older middle ground: the task exits and is re-queued every poke interval.
`deferrable=True` is better where the operator supports it — but it needs a **triggerer** process running.

Plenty of non-sensor operators take `deferrable=True` too — anything that mostly sits waiting on a remote system.



Assets: let the **data** trigger the next DAG

Producer — declares an outlet

```
from airflow.sdk import Asset

sales = Asset("s3://warehouse/sales")

with DAG(dag_id="ingest", schedule="@daily"):
    PythonOperator(
        task_id="load_sales",
        python_callable=load,
        outlets=[sales],
    )
```

Consumer — schedules on it

```
with DAG(dag_id="reporting", schedule=[sales]):
    PythonOperator(
        task_id="report",
        python_callable=build_report,
    )
```

Runs when **load_sales** succeeds. No cron, no aligned start dates, no sensor holding a slot.



Airflow 3 renamed Datasets to **Assets** and added an `@asset` decorator plus an Assets view in the UI. An **AssetWatcher** can trigger on an external message queue — event-driven, with no sensor at all.



Exercises 8 & 9

8 · reporting_sensor

Cross-DAG dependencies with

`ExternalTaskSensor` — why schedules and start dates must line up

`8_reporting_sensor`

9 ·

reporting_asset_dependence

Event-driven scheduling with Assets and outlets, contrasted with interval-based scheduling

`9_reporting_asset_dependence`



Sharing data & configuration



Sharing **data** between tasks

From the Airflow docs on operators: "in general, if two operators need to share information, like a filename or small amount of data, you should consider combining them into a single operator."



🤔 If there's no other way, use the XCom mechanism. Note it's stored in the metadata database — consider size and serializability ("picklability") before using it.



Most of the time, pass data between tasks via an external storage mechanism, like cloud blob storage.



Variables and connections: runtime-dependent, global configuration



Do not use them to pass data from one task to another — use XComs instead, or better, store data externally.

	Metastore DB	Environment variables	Secrets backend
Create / read / update / delete	Web UI, CLI, programmatically	<code>export</code> <code>AIRFLOW_VAR_F00=BAR</code>	Depends on backend
Priority	3	2	1
Notes		Won't show in the UI	Won't show in the UI; also works for config settings via the <code>_secret</code> suffix



Exercises 10 & 11

10 · xcoms

Passing data via XCom and dynamic task mapping
(`.expand()`) — classic API vs. TaskFlow API

```
10_xcoms_classicapi /  
10_xcoms_taskflowapi
```

11 · connections_and_hooks

Airflow Connections & `PostgresHook` — why
hardcoded credentials in a DAG file are a security
finding

```
11_connections_and_hooks
```



Best practices



Airflow best practices

- Don't perform time-consuming operations at the top level. The DAG processor shouldn't make database connections, run simulations, or transform tables — put that work in operators
- **Create idempotent workflows.** You should be able to "time travel" and rerun jobs weeks after their intended trigger moment, as if they ran on the day they had to. Use upserts instead of inserts, and template on the logical date rather than `datetime.now()`
- Idempotency is what makes **retries** safe. A task that is *not* idempotent is the one that needs `retries=0`
- Each DAG has an owner / responsible
- Specify timezone to prevent time-zone issues: `pendulum.datetime(2026, 7, 1, tz="Europe/Paris")`
- Only a very limited amount of data can be shared between operators via XComs — if two operators need to share information, consider combining them into one operator, or communicating via shared files



Retries, timeouts and telling someone it **broke**

```
from datetime import timedelta
from airflow.providers.smtp.notifications.smtp \
    import Smtplib

with DAG(
    dag_id="reporting",
    default_args={
        "retries": 2,
        "retry_delay": timedelta(minutes=5),
        "retry_exponential_backoff": True,
        "execution_timeout": timedelta(hours=1),
    },
    on_failure_callback=Smtplib(
        to="data-team@example.com",
        subject="[Airflow] {{ dag.dag_id }} failed",
    ),
):
    ...
```

- `retries` — safe precisely because the task is idempotent. Transient network blips are the common case
- `retry_exponential_backoff` — stop hammering a service that is already struggling
- `execution_timeout` — without it, a hung task waits forever and holds its slot
- **Notifiers** — `on_failure_callback` takes a notifier object. SMTP, Slack, PagerDuty, ... This replaced the old `EmailOperator`, and it fires wherever the failure happens



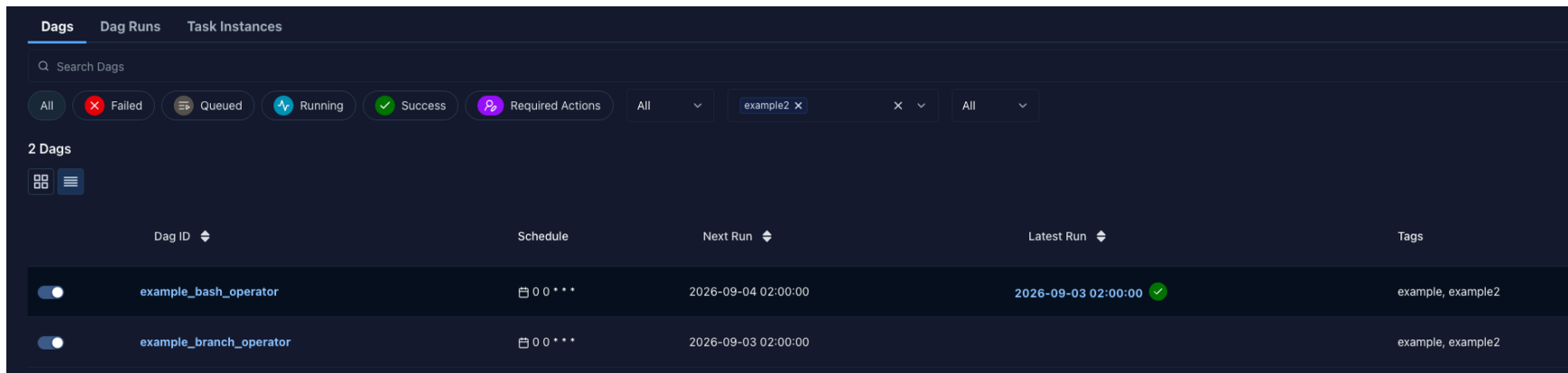
A DAG nobody is alerted about is a DAG nobody notices has been failing for three weeks.



Documentation: add tags

```
dag = DAG(  
    dag_id="example_bash_operator",  
    schedule="0 0 * * *",  
    tags=["example", "example2"],  
)
```

💡 Use department, team name, distribution group, or project/product name as tags.



	Dag ID	Schedule	Next Run	Latest Run	Tags
☒	example_bash_operator	0 0 * * *	2026-09-04 02:00:00	2026-09-03 02:00:00 ✓	example, example2
☒	example_branch_operator	0 0 * * *	2026-09-03 02:00:00		example, example2



Filter the DAGs list by tag, then share the URL — the Airflow 3 UI keeps the filter in the query string.



Always specify your DAG's timezone for peace of mind around **DST**

```
dag = DAG("timezone aware DAG", description="This DAG observes daylight saving time.",  
         start_date=pendulum.datetime(2026, 1, 1, tz="Europe/Brussels"))
```

UI set to UTC

10:04 UTC AA

Schedule: */5 * * * * Next Run: 2022-06-10, 10:00:00

Run After: 2022-06-10, 10:05:00 UTC

Data Interval
Start: 2022-06-10, 10:00:00 UTC
End: 2022-06-10, 10:05:00 UTC

UI set to CEST (+02:00)

12:03 CEST (+02:00) AA

Schedule: */5 * * * * Next Run: 2022-06-10, 12:00:00

Run After: 2022-06-10, 12:05:00 CEST

Data Interval
Start: 2022-06-10, 12:00:00 CEST
End: 2022-06-10, 12:05:00 CEST



Same run, two display timezones. Use a `pendulum` datetime instance, not one from the builtin `datetime` module.



Managed Airflow



MWAA



Astronomer



Composer



Conveyor

Managed means someone else runs the scheduler, the database and the upgrades — you still write the DAGs.



The question to ask any of them in 2026: **which Airflow version do you actually offer, and how long after an upstream release?** Airflow 3 was a major migration, and the managed offerings did not all arrive at once.



Wrap-up



Three things to remember

1

KEEP THE TOP LEVEL CHEAP

The scheduler reparses your DAG file constantly. No database connections, no simulations, no table transformations at import time — put that work in operators.

2

AIRFLOW IS CONFIG AS CODE

Use the constructs Python and Airflow give you — for-loops, imports, `TaskGroup`, `BranchOperator` — to keep workflows modular and DRY.

3

MAKE EVERY RUN IDEMPOTENT

You should be able to time-travel and rerun a job weeks after its intended trigger moment and get the same result. Template on the logical date, never on `datetime.now()`.



Airflow: what's next?

Go deeper on what we covered

- `dag.test()` — run a whole DAG in one Python process, no scheduler, straight in your debugger
- Custom operators and hooks
- Testing and DAG validation in CI
- Dynamic DAG creation, e.g. from YAML

Newer Airflow 3 ground

- **Human-in-the-loop** (3.1) — approve, reject or pick a branch mid-run
- `@asset` and asset watchers for event-driven pipelines
- **DAG versioning** and DAG bundles
- Edge executor, and multi-team deployments

Plus the operational half: hosting and executors, CI/CD, secrets, monitoring, lineage.





Thank you!

Questions?

